Monte Carlo Control

This notebook illustrates the basic algorithms of Monte Carlo control methods using the classical "gridworld" environment. We consider the case where the agent does not know the transition probabilities and has to learn them from experience.

The general idea is to perform a Generalized Policy Iteration (GPI) algorithm, where we alternate between policy evaluation and policy improvement steps.

Monte Carlo Estimation of Action-Values (Policy Evaluation)

See Sutton & Barto, Chapter 5.1 and 5.2. for motivating the approach. The key points are:

- In the absence of a model, it is necessary to estimate $Q_\pi(s, a)$, the expected return starting from state s, taking action a and following policy π thereafter. Estimating the value function $V_\pi(s)$ is not sufficient, because we need to know the value of taking action a in state s to be able to improve the policy.
- The value of a state-action pair is estimated by simulating many episodes of a policy and averaging the returns of all episodes that start from that state-action pair.
- There are two variants of Monte Carlo estimation: first-visit and every-visit. Both converge to the true value of the state-action pair under the policy π.
- The first-visit MC method estimates the value of a state-action pair as the average of the returns that followed the first visit to it.
- The every-visit MC method estimates the value of a state-action pair as the average of the returns that followed all the visits to it.
- We need to estimate the value of each state-action pair, not just the ones that seem favorable, this is the general requirement of maintaining exploration. We can do this by randomly choosing the initial state-action pair and then following the policy π until the end of the episode.

If N episodes are generated, the average return for each state-action pair, $Q(s,a)$ is calculated as follows.

$$Q(s,a) = \frac{1}{N(s,a)} \sum_{k=1}^{N(s,a)} G_k(s,a)$$

where $N(s,a)$ is the number of episodes that visit state-action pair (s,a) at least once , and $G_k(s,a)$ is the return following the first visit to state-action pair (s,a) in the k-th episode. Let $t_k(s,a)$ be the first time in episode k that state-action pair (s,a) is visited, and T_k be the length of the k-th episode. $G_k(s,a)$ is the sum of the discounted rewards of this episode, starting from time step $t_k(s,a)+1$ to T_k.

$$G_k(s,a) = R_{t_k(s,a)+1} + \gamma R_{t_k(s,a)+2} + \cdots + \gamma^{T_k - t_k(s,a)-1} R_{T_k}$$

The average over the episodes is computed incrementally. To implify notation, we concentrate on a single state-action pair (s,a). Let Q_n be the estimate of the state-action value after $n-1$ first visits.

$$Q_n = \frac{1}{n-1} \sum_{k=1}^{n-1} G_k$$

After another visit to the same state-action pair, we have

$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{k=1}^{n} G_k \\ &= \frac{1}{n} \left(\sum_{k=1}^{n-1} G_k + G_n \right) \\ &= \frac{1}{n} \left((n-1) Q_n + G_n \right) \\ &= Q_n + \frac{1}{n} (G_n - Q_n) \end{aligned}$$

Policy Improvement

Policy improvement is performed by making the policy greedy with respect to the current action-value function $Q(s,a)$.

$$\pi(s) = \arg\max_a Q(s,a)$$

The policy improvement interation constructs a new policy π_{k+1} as the greedy policy with respect to $Q_{\pi_k}(s, a)$, and we have:

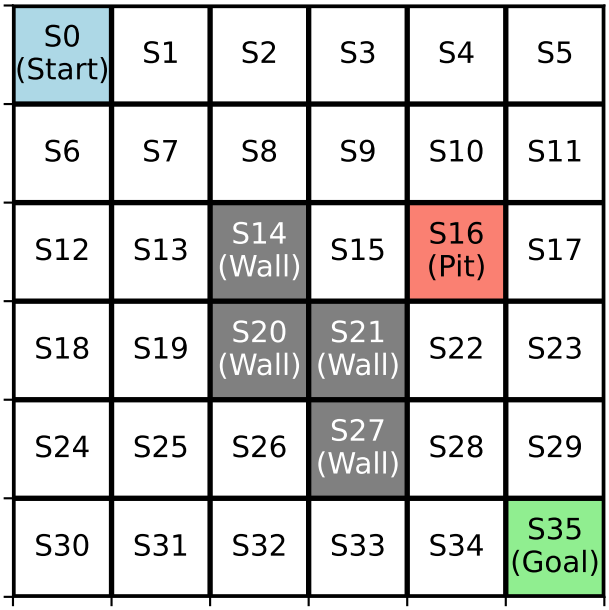
$$\begin{aligned} Q_{\pi_k}(s, \pi_{k+1}(s)) &= \max_a Q_{\pi_k}(s, a) \\ &\geq Q_{\pi_k}(s, \pi_k(s)) \\ &\geq V_{\pi_k}(s) \end{aligned}$$

The policy improvement theorem insures that each π_{k+1} is uniformly better or equal to π_k. In practice, we do not attempt to get a complete policy evaluation before initiating a policy improvement step. Instead, it is sufficient to alternate between policy evaluation and policy improvement at every episode. We demonstrate this concept with an algorithm called "Monte Carlo with Exploring Starts".

The test case

The environment is a 6x6 grid, where the agent starts at the top-left corner (state S_0) and aims to reach the bottom-right corner (state S_{35}). The agent can move in four directions: up, down, left, and right. However, there are obstacles (walls) at states S_{14}, S_{20}, S_{21} and S_{27} that the agent cannot pass through, and a pit at state S_{16} where the agent would get stuck. The objective is modeled by assigning a reward to each state: the pit has a reward of -1000, and the end goal a reward of 10. Passing through any other state has a reward of -1. The agent receives the reward immediately upon entering a state. The episode ends when the agent reaches either the end goal or the pit. Maximixing the reward is equivalent to finding the shortest feasible path to the goal.

Gridworld with obstacles and a pit



The moves are not fully predictable. The agent intends to move in a certain direction, but the actual move is subject to some randomness. The transition probabilities are given by the following table, but the agent does not know them:

	Probability of move			
Intended Action	up	down	left	right
Up	80%	0%	10%	10%
Down	0%	80%	10%	10%
Left	10%	10%	80%	0%
Right	10%	10%	0%	80%

Monte Carlo with Exploring Starts

The first component of the algorithm is the generation of episodes, given a policy.

```
ACTIONS = [(-1,0), (1,0), (0,-1), (0,1)]  # Up, Down, Left, Right

slippage = {
        0: [0, 2, 3], # Up -> Up, Left, Right
        1: [1, 2, 3], # Down -> Down, Left, Right
        2: [2, 0, 1], # Left -> Left, Up, Down
```

```
        3: [3, 0, 1]  # Right -> Right, Up, Down
}

# probabilities of move consistent with action and of slippage
probs = [0.8, 0.1, 0.1]

def move(s, a_idx):
    """
    Returns the next state and reward for an action a_idx in state s.
    """
    row, col = divmod(s, GRID_SIZE)
    dr, dc = ACTIONS[a_idx]
    nr = max(0, min(GRID_SIZE-1, row + dr))
    nc = max(0, min(GRID_SIZE-1, col + dc))
    next_s = nr * GRID_SIZE + nc

    if next_s in OBSTACLES:
        next_s = s

    if next_s in GOAL_STATES:
        reward = GOAL_REWARD
    elif next_s in PIT_STATES:
        reward = PIT_REWARD
    else:
        reward = -1

    return next_s, reward


def get_transition_results(s, a_idx):
    """
    Returns the actual result of an action, taking slippage into account.
    """
    if s in GOAL_STATES or s in PIT_STATES or s in OBSTACLES:
        return s, 0

    # Sample slippage
    idx = np.random.choice(len(slippage[a_idx]), p=probs)
    return move(s, slippage[a_idx][idx])


def generate_episode(policy, initial_state, initial_action=None, max_steps=100):
```

```
    """
    Generates an episode using a given policy starting from initial_state.
    policy: an array of length len(STATES) with values 0 to 3.
    initial_action: Optional first action to take for exploring starts.
    """
    current_state = initial_state
    episode = []

    for step in range(max_steps):
        # Follow the policy or exploring start
        if step == 0 and initial_action is not None:
            action = int(initial_action)
        else:
            action = int(policy[current_state])

        # Determine the outcome based on environment's transition probabilities
        next_state, reward = get_transition_results(current_state, action)

        # Record sequence: state, action, reward
        episode.append((current_state, action, next_state, reward))

        if next_state in GOAL_STATES or next_state in PIT_STATES:
            break

        current_state = int(next_state)

    return episode
```

As an illustration, let's generate three episodes starting from random initial states.

```
np.random.seed(314)
random_policy = np.random.choice(4, size=STATES)

# For Monte Carlo with Exploring Starts, we can start at a random valid state
valid_states = [s for s in range(STATES) if s not in GOAL_STATES and
                s not in PIT_STATES and s not in OBSTACLES]

# Start randomly for exploring starts
initial_states = np.random.choice(valid_states, size=3, replace=False)
# Generate 3 sample paths
episodes = [generate_episode(random_policy, i_s) for i_s in initial_states]
```

The following table summarizes the experiment: two episodes ended in terminal states, while the other one completed the intended 100 steps. The figure below illustrates the paths. The policy is represented by an arrow in each state.

Table 1

Path	Initial State	Final State	Length (Steps)
1	S_{26}	S_{16}	53
2	S_{28}	S_{35}	2
3	S_1	S_8	100

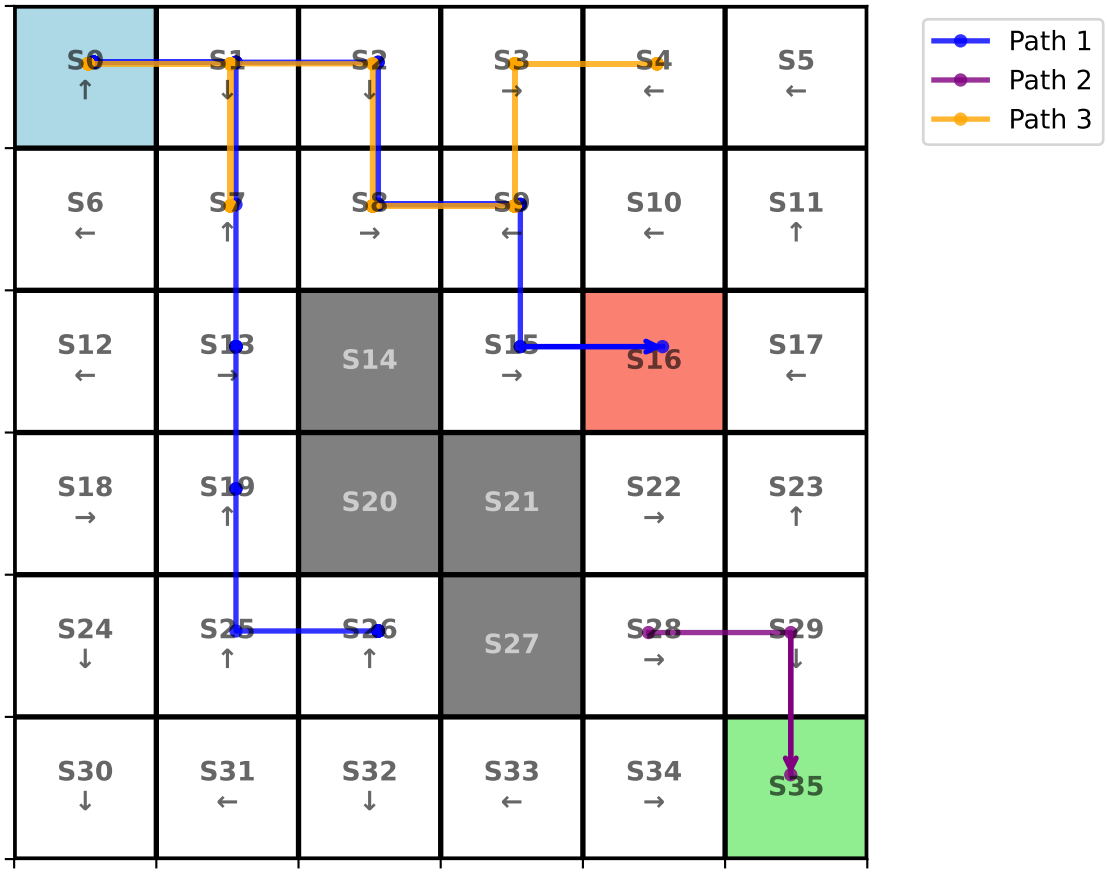


Figure 1: Three sample paths starting from random initial states. The policy is represented by an arrow in each state.

The Complete Algorithm: Monte Carlo with Exploring Starts

The complete algorithm alternates between episode generation (evaluation) and policy improvement (Sutton and Barto, p. 99)

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$
Initialize:

- $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
- $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
- $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):

- Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
- Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, S_1, A_1, \ldots, S_{T-1}, A_{T-1}, R_T$
- $G \leftarrow 0$
- Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
  - $G \leftarrow \gamma G + R_{t+1}$
  - Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
    - Append G to $Returns(S_t, A_t)$
    - $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
    - $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

```
def mc_exploring_starts(num_episodes, gamma=0.9):
    # Initialize Q(S,A) arbitrarily
    Q = np.zeros((STATES, 4))

    # We will use incremental average instead of keeping all returns
    returns_count = np.zeros((STATES, 4))

    # Starting policy
    policy = np.random.choice(4, size=STATES)

    # start from any valid state
    valid_states = [s for s in range(STATES) if s not in GOAL_STATES and
                    s not in PIT_STATES and s not in OBSTACLES]

    for _ in range(num_episodes):
        # Exploring start
        start_state = np.random.choice(valid_states)
        start_action = np.random.choice(4)
```

```
        # Generate an episode
        episode = generate_episode(policy, start_state, initial_action=start_action)

        # Collect all state-action pairs in the episode to find first visits
        sa_pairs = [(step[0], step[1]) for step in episode]
        first_visit_indices = {}
        for t, sa in enumerate(sa_pairs):
            if sa not in first_visit_indices:
                first_visit_indices[sa] = t

        # Now go backwards to compute G
        G = 0
        for t in reversed(range(len(episode))):
            s, a, next_s, r = episode[t]
            G = gamma * G + r

            # If this is the first visit
            if first_visit_indices[(s, a)] == t:
                returns_count[s, a] += 1
                # Incremental mean update: Q_{n+1} = Q_n + (1/n) * (G - Q_n)
                Q[s, a] = Q[s, a] + (1.0 / returns_count[s, a]) * (G - Q[s, a])

                # Policy improvement
                policy[s] = np.argmax(Q[s])

    return Q, policy

np.random.seed(42)
Q_optimal, optimal_policy = mc_exploring_starts(num_episodes=100000)
```

The figure below shows the optimal policy found, which is essentially the same as the one in Figure **??**. The crucial observation is that the agent corectly learned the strategy to avoid the pit.

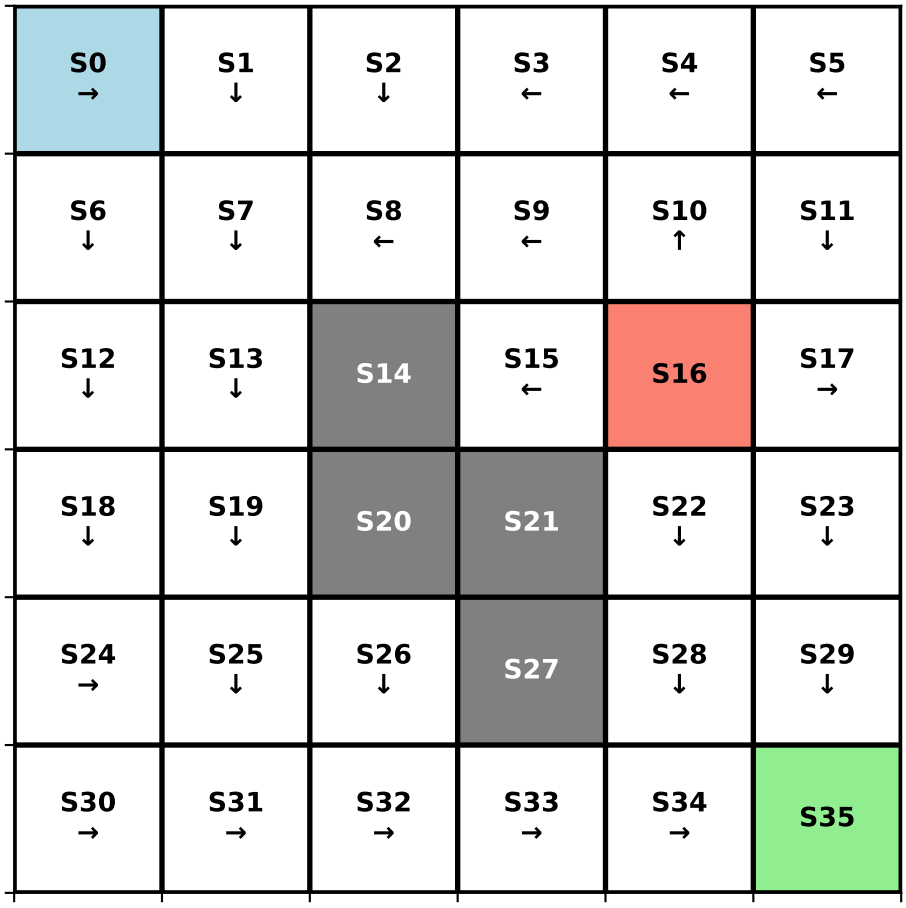


Figure 2: Optimal policy found by Monte Carlo with Exploring Starts